

Дилемата на двойния агент

- **Времево ограничение:** 12 минути.
- **Дисково пространство:** 5 GB
- **Среда:** едно GPU (≈ 16 GB VRAM), без интернет
- **Размер на решението:** `solution.ipynb` ≤ 1 MB
- **Базов резултат:** 0

В националния център по изкуствен интелект в Астана два компютърни модела — Model R (ResNet-18) и Model V (ViT-Tiny) — анализират снимки. В момента и двата модела се справят безупречно, постигат 100% точност и дават еднакви отговори за всяко отделно изображение. За да провери колко различни всъщност са техните умни „мозъци“, главният учен Ви поставя предизвикателство: направете малки, почти невидими промени в пикселите на всяка снимка, така че Model R и Model V да бъдат в пълно несъгласие.



1. Задача

Два предварително обучени класификатора на изображения разглеждат едно и също изображение. Върху изображенията, предоставени в тази задача, и двата класификатора постигат 100% точност.

- **Model R:** `torchvision.models.resnet18` (CNN, ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (Transformer, ViT-Tiny).

Вашата задача е да създадете малка промяна („смущение“) за всяко изображение, така че двата модела да не са съгласни. За всяко изображение трябва да създадете **две различни** смущения:

- **Тип A:** след добавянето му Model R продължава да класифицира изображението правилно, но Model V го класифицира неправилно.
- **Тип B:** след добавянето му Model V продължава да класифицира изображението правилно, но Model R го класифицира неправилно.

Всяко смущение трябва да бъде достатъчно *малко*, за да се забелязва трудно. По-малките смущения получават по-висок резултат (вижте Раздел 5). Смущението се прилага директно към оригиналното изображение на ниво пиксели.

2. Публични данни

Заедно със задачата е предоставен набор от изображения, организиран в два дяла — `train` (100 изображения) и `test_public` (100 изображения) — всеки от които съдържа изображения с различна резолюция. Всички изображения са от 1000-те класа на ImageNet-1K и както Model R, така и Model V постигат 100% точност и върху двата дяла.

Предоставени са следните файлове:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

По време на оценяването Вашата папка `dataset/test_public/` прозрачно се заменя с два скрити набора от изображения (`test_leaderboard_a` и `test_leaderboard_b`) за официалното оценяване. Всеки от тях съдържа **100 изображения** във формат PNG и файл с етикети.

Забележка: За тази задача етикетите в тестовите набори от данни са достъпни.

3. Формат на изхода

За всяко изображение трябва да създадете два файла:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- $\{index\}$ (0, 1, 2, ...), съответства на името на изображението в наборите от данни.
- Всеки файл представлява един тензор, запазен чрез `torch.save`. Формата му трябва да бъде $3 \times H \times W$, където H и W съответстват на **оригиналната** резолюция на това изображение (а не на 224×224).
- Кодът трябва да създаде само един ZIP файл, `submission.zip`. Поставете всички `.pt` файлове на най-горното ниво на ZIP архива, без обхващаща папка или поддиректории.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Кодът в тетрадката ще Ви предупреди, ако има каквито и да било проблеми с изходния формат.

4. Ограничения

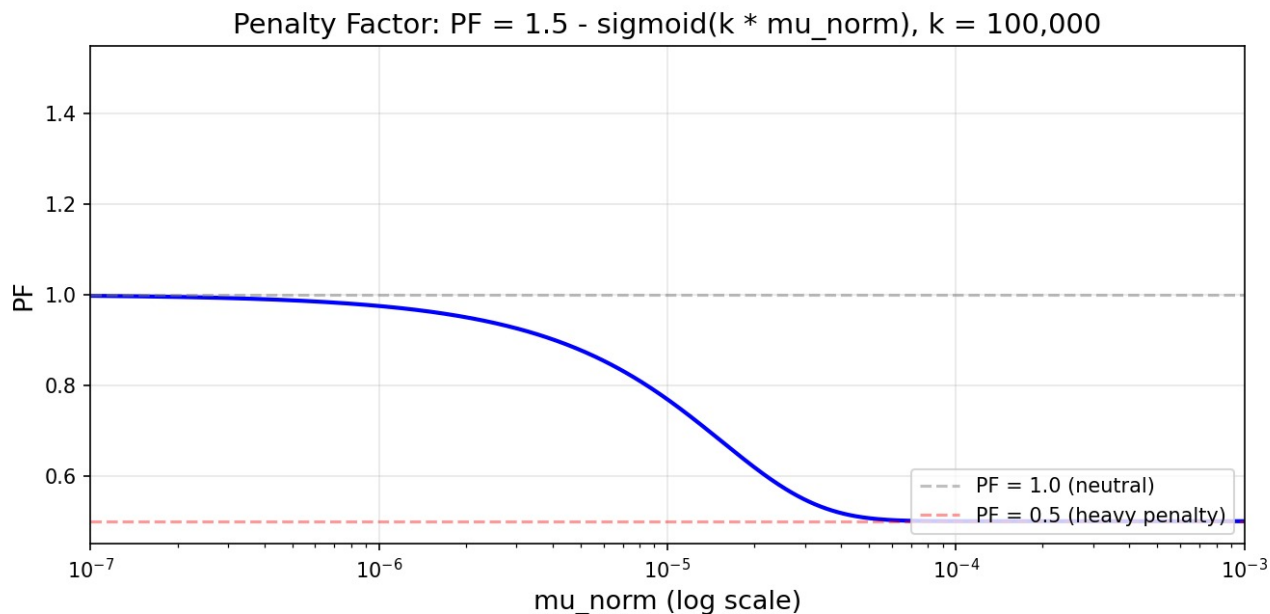
- **Модели:** Трябва да използвате `torchvision.models.resnet18(pretrained=True)` и `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. Не са разрешени други предварително обучени модели.
- **Конвейер от трансформации (прилага се принудително при оценяването):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. Вижте Секция 3 от `baseline.ipynb` за подробности.
- **Резолюция на смущението:** Трябва да съответства на **оригиналната** резолюция на необработеното изображение (а не на 224×224). Тензорът се добавя към необработеното изображение *преди* конвейера от трансформации.
- **Формат на изхода:** Само `.pt` файлове — без PNG/JPG. Тензорите се добавят към необработеното изображение и стойностите на пикселите се ограничават до `[0, 1]` преди предварителната обработка.
- **Именуване на файловете:** Плоска структура, стриктен формат `{index}_a.pt / {index}_b.pt`. Без поддиректории в zip архива.
- **Библиотеки:** `torch, torchvision, timm`.

5. Оценяване

Крайният резултат се изчислява по следния начин. Нека M е броят на изображенията в дяла, $Score_A$ — броят на успешните смущения от Тип A, а $Score_B$ — броят на успешните смущения от Тип B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF е функция, предназначена да наказва смущенията с висока норма и да бъде много чувствителна в близост до горната граница на резултата. Тя е ограничена в диапазона от 0.5 до 1. Пълната имплементация може да бъде видяна в Секция 8 на `solution.ipynb`.



Фигура: Кривата на наказателната функция.

6. Проверка на предаването

В тетрадката има проверки, които Ви предупреждават, ако има проблеми с форматирането, в Раздел 7 на `solution.ipynb`.

7. Локално тестване

`solution.ipynb` съдържа пълен работещ пример. Той зарежда публичните данни, двата модела и официалната функция за оценяване и записва ZIP файл за предаване. Прочетете го, преди да започнете.

8. Начин на предаване

- Запазете промените си в `solution.ipynb`.
- Отворете раздела Git в лявата странична лента на JupyterLab.
- Изпълнете **Stage** за `solution.ipynb` (иконата + до него).
- Въведете съобщение за commit и щракнете върху **Commit**.
- Щракнете върху облака със стрелка нагоре, за да изпълните push.
- Върнете се на тази страница на състезанието и щракнете върху **Submit**.

Предайте точно един файл с име `solution.ipynb`.